

1 • PREPARE ONCE (OFFLINE SCRIPTS)

Source dataset
coursera_course_2024.csv, 6,645 rows, 14 columns. Pinned Hugging Face revision, SHA-256 checked by fetch_inputs.py.

↓

prepare_data.py • clean
Drop 3 exact duplicates; stable IDs from canonical URLs
Missing stays missing (no rating ≠ 0, no level ≠ Beginner)
Safe skill-list parsing; 9,134 labels → canonical skills
196 descriptions marked Suspect (shared or off-topic)

↓

build_index.py • embed
all-MiniLM-L6-v2 (pinned), title + skills + validated description → 384-d unit vectors. Writes a manifest of hashes.

↓

Knowledge base data/processed/
courses.json 6,642 courses · embeddings.npy 6,642 × 384 ·
course_ids.json · manifest.json

↓

Reviewed rules data/rules/
Skill aliases · 3 tracks, 21 skills, 13 dependencies with reasons · 8 course prerequisites quoted from descriptions

→ loaded and verified at startup

→ rules

2 • FASTAPI SERVICE, ONE PROCESS • POST /RECOMMEND (ALSO USED FOR WHAT-IF)

Startup: check artefact hashes (refuse a mismatch) · load catalog · build BM25 index · load model locally. Model missing → labelled keyword-only mode.

1

Read the goal
Clauses → known, negated and target skills; track detection (or ask when ambiguous). Chip edits win; simulated What-If skills kept separate.

2

Filter
Difficulty, organization, minimum rating → eligible courses. Unknown values never pass a filter.

3

BM25 keywords
Top 50; hits sharing no query term dropped

MiniLM cosine
Top 50 above similarity 0.30

RRF fusion
 $\Sigma 1/(60 + \text{rank})$, deterministic ties

4

Skill gap
Track goal skills + prerequisite closure – known (– simulated) → missing goal skills and foundations; coverage now and after the path.

5

Learning path
Greedy over the whole filtered catalog: learnable skills first, modelled prerequisites respected, ≤ 5 steps, uncovered skills reported with a reason.

6

Personalise top 10
Prerequisites met → unknown → unmet; then missing goal skills taught; then fused rank; rating breaks ties. Return ≤ 5.

7

Honest Advisor
Templated from the response: why this helps · before you start (with prerequisite source) · things to consider.

POST /feedback
5 labels + note → SQLite (parameterised). Stored for review, not auto-training.

POST /search
Raw bm25 / semantic / hybrid for evaluation

GET /health · /catalog · /evaluation
Readiness and mode · tracks and filters · saved report

3 • REACT APP (VITE, TYPESCRIPT)

Discover
Goal in plain language, three examples
Editable skill chips, track, filters
Skill chart: known · missing goal skills · missing foundations
Coverage now and after the path
Numbered learning path with prerequisite sources
Up to 5 courses with Honest Advisor and feedback

What-If
Press a missing tile → same /recommend with simulated_skills. Baseline kept separately; shows skills, steps added, moved or dropped, or "No path change". Reset restores the profile.

Evaluation screen
Reads the saved report only; never runs models. Shows "Not evaluated yet" until a real report exists.

SQLite feedback.sqlite3
Course, label, note, request and profile context, timestamp

→
←
JSON over /api proxy

→
←
feedback

4 • EVALUATION (SCRIPTS/EVALUATE.PY) USES THE SAME ENGINE AND THE SAME /RECOMMEND PIPELINE

Frozen inputs
18 goals (6 dev, 12 held out) · 9 profiles with hand-written expected gaps · 5 edge cases

→

Blind pooled judgments
Top 5 of BM25, semantic and hybrid, method hidden; 187 labels with reasons

→

Measure
Precision@5, MRR@5 · gap P/R/F1 · path checks · warm latency

→

report.json
→ Evaluation screen, with findings and limitations